CS-2001
Data Structures
Fall 2023

Hashing

Rizwan Ul Haq

Assistant Professor

FAST-NU

rizwan.haq@nu.edu.pk

Hashing

- Tables
- Direct address tables
- Hash tables
- Hash Functions
- Collision and collision resolution
- Chaining

Introduction

- Many applications require a dynamic set that supports dictionary operations
- Example: a compiler maintaining a symbol table where keys correspond to identifiers
- Hash table is a good data structure for implementing dictionary operations
- Although searching can take as long as a linked list implementation i.e. O(n) in worst case

Introduction

- With reasonable assumptions it can take O(1) time
- In practice hashing performs “extremely” well
- A hash table is a generalization of an ordinary array where direct addressing takes O(1) time
- When the actual keys is NOT small relative to the total number of keys, hashing is an effective alternative
- A key can be accessed using an array index, or is computed

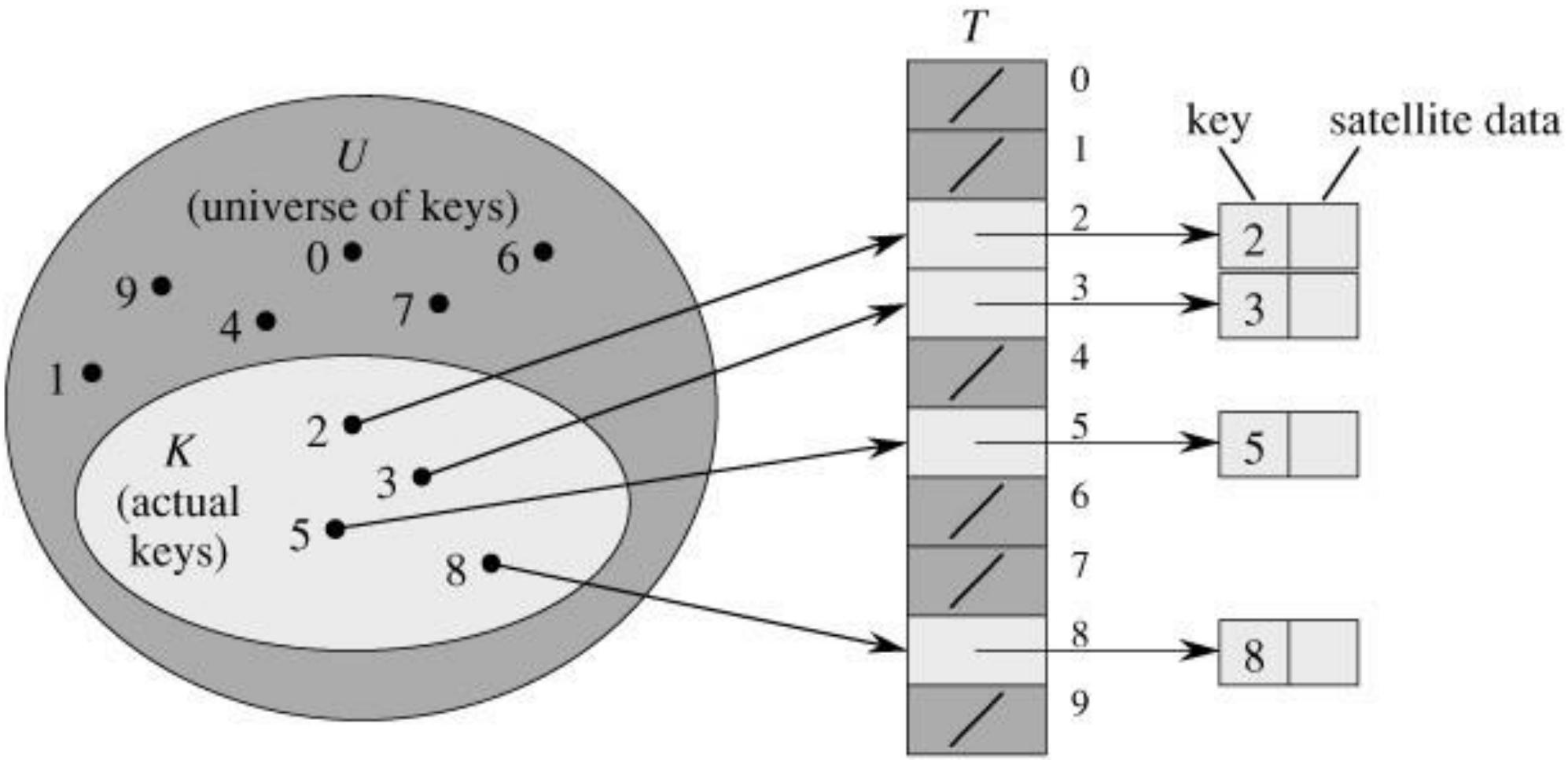
What are Tables?

- Table is an abstract storage device that contains table entries
- Each table entry contains a unique key ***k***.
- Each table entry may also contain some information, associated with its key.
- A table entry is an ordered pair ***(K, I)***

Direct Addressing

- Suppose:
  - The range of keys is 0..*m*-1
  - Keys are distinct
- The idea:
  - Set up an array T[0..m-1] in which
    - T[*i*] = *x* if $x \in T$ and key[*x*] = *i*
    - T[*i*] = NULL otherwise
  - This is called a *direct-address table*
    - Operations take O(1) time!

Direct Access Table



Ref: https://slideplayer.com/slide/1649208/

Advantages with Direct Addressing

- Direct Addressing is the most efficient way to access the data since,
  - It takes only single step for any operation on direct address table
  - It works well when the Universe U of keys is reasonable small

Difficulty with Direct Addressing

When the universe *U* is very large…

- Storing a table *T* of size *U* may be impractical, given the memory available on a typical computer
- The set *K* of the keys actually stored may be so small relative to *U* that most of the space allocated for *T* would be wasted

An Example

- **A table:** 50 students in a class
- **The key:** 13 digit ***CNIC***, used to identify each student
- Number of different 13 digit number = 10^{13}
- The fraction of actual keys needed. $50/10^3$, **0.0000000005%**
- Percent of the memory allocated for table wasted, **99.9999999995%**

An ideal table needed!

- The table should be of small fixed size.
- Any key in the universe should be able to be mapped in a slot into the table, using some mapping function

Hash Tables

- **Definition:** The ideal table data structure is merely an array of some fixed size, containing the elements

- **Consist:** An **array** and a **mapping function** (known as hash function)

- Used for performing insertion, deletion and lookup on average in constant time

Basic Idea

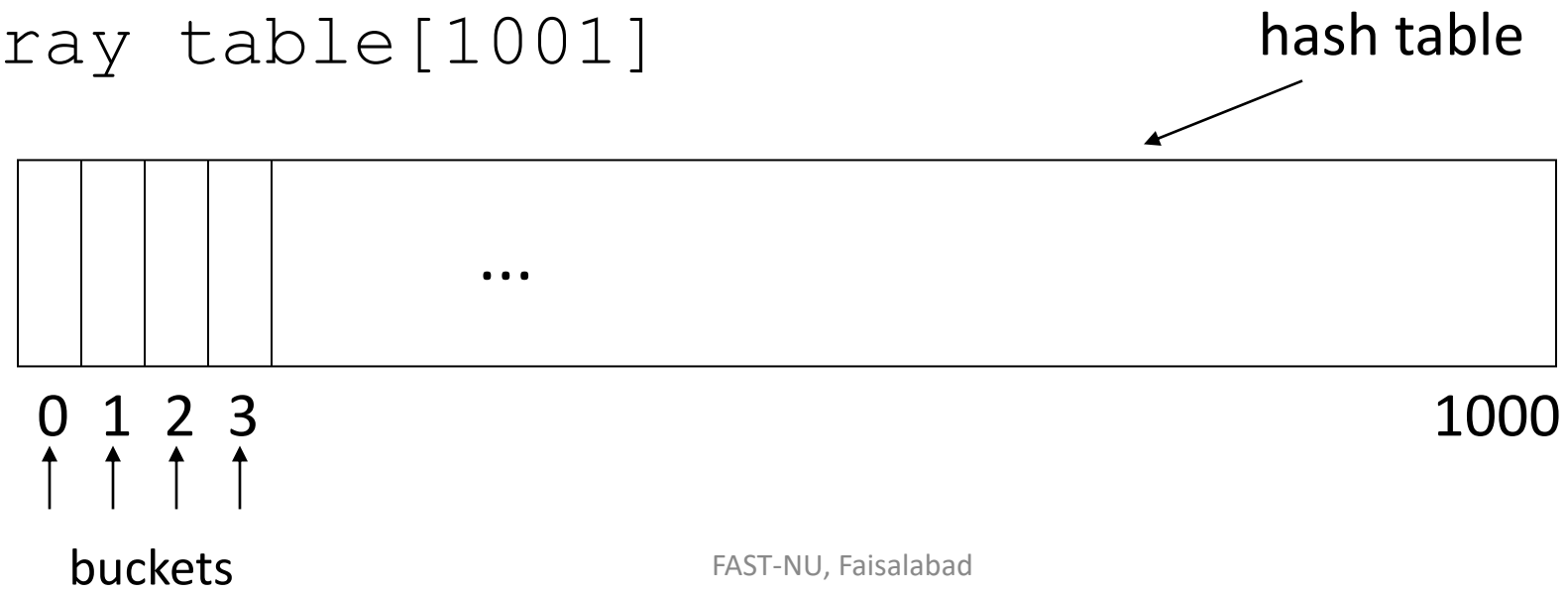
- Use hash function to map keys into positions in a hash table

Ideally

- If element ***e*** has key ***k*** and ***h*** is hash function, then ***e*** is stored in position ***h(k)*** of table i.e. HT[*j(k)*]
- To search for ***e***, compute ***h(k)*** to locate position. If no element, dictionary does not contain ***e***

Example

- Dictionary Employee Records
  - Keys are ID numbers (951000 - 952000), no more than 100 students
  - Hash function: h(k) = k-951000 maps ID into distinct table positions 0-1000
  - `array table[1001]`



Analysis (Ideal Case)

- O(b) time to initialize hash table (b number of positions or buckets in hash table)
- O(1) time to perform *insert, remove, search*

Ideal Case is Unrealistic

- Works for implementing dictionaries, but many applications have key ranges that are too large to have 1-1 mapping between buckets and keys!

Example:

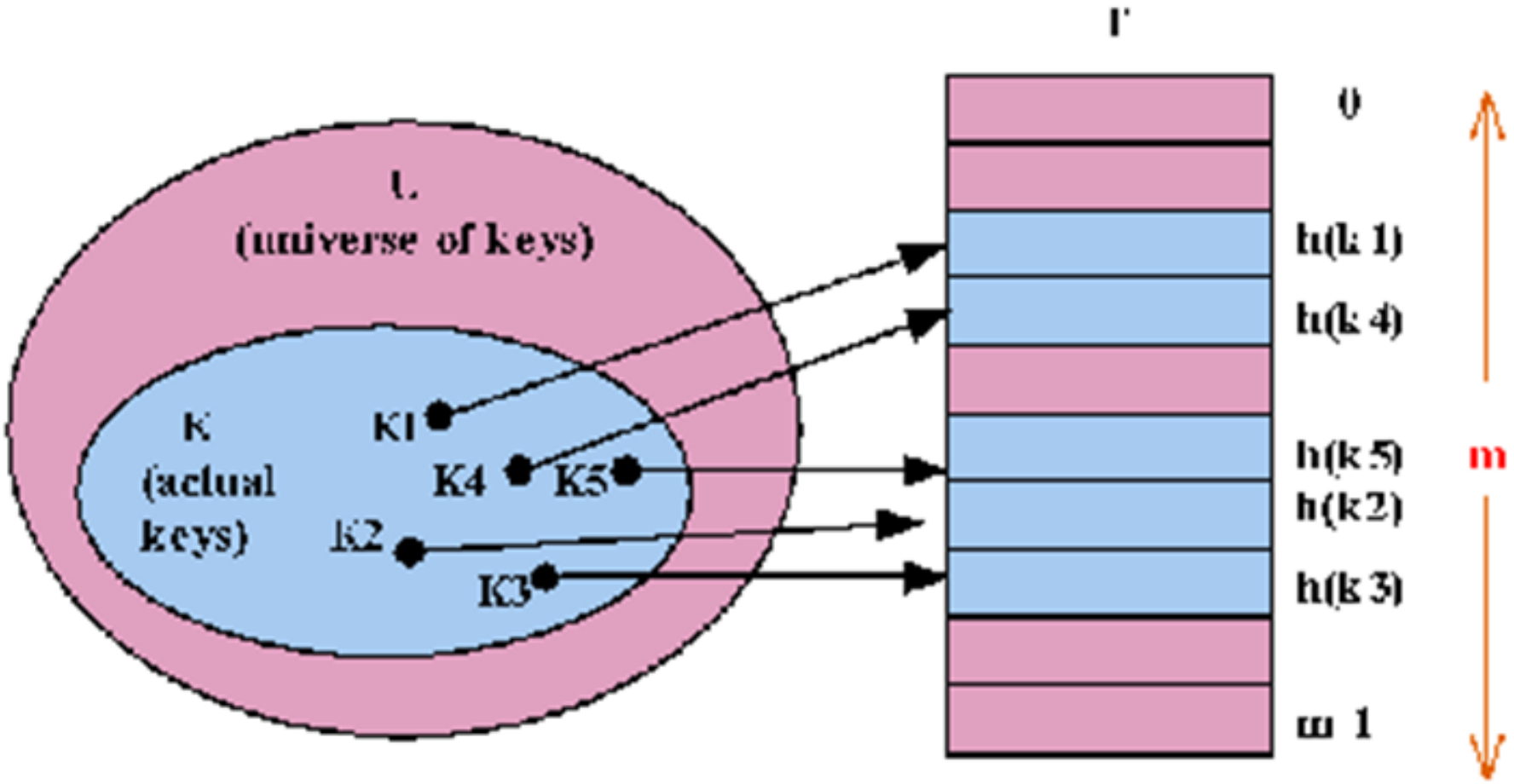
- Suppose key can take on values from 0 … 65,535 (2 byte unsigned int)
- Expect ≈ 1,000 records at any given time
- Impractical to use hash table with 65,536 slots!

Compared to direct addressing

- Advantage: Requires less storage and runs in *O(1)* time.
- Comparison

	Storage Space	**Storing k**
Direct Addressing	|U|	Store in slot k
Hashing	m	Store in slot h(k)

Hash Table: Ideal



Hash Functions

- If key range is too large, using hash table with fewer buckets, a hash function can map multiple keys to the same bucket:

  $h(k_1) = \beta = h(k_2)$: k_1 and k_2 have collision at slot β

- Popular hash functions: hashing by division

  h(k) = k % D, where D number of buckets in hash table

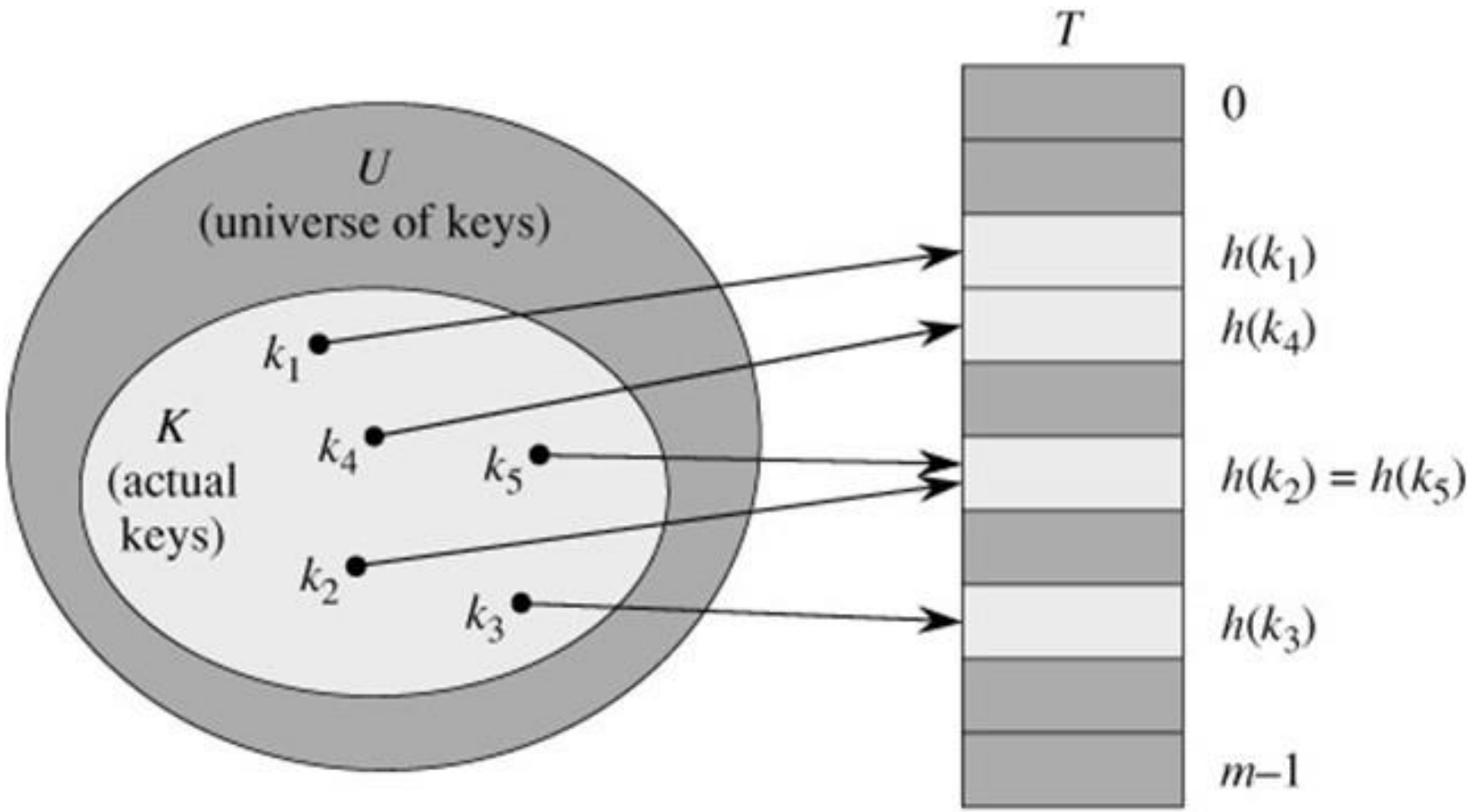
- Example: hash table with 11 buckets

  h(k) = k%11

  80 $\rightarrow$ 3 (80%11= 3), 40 $\rightarrow$ 7, 65 $\rightarrow$ 10

  58 $\rightarrow$ 3 collision!

Hash Table: Collision



Collision Resolution Policies

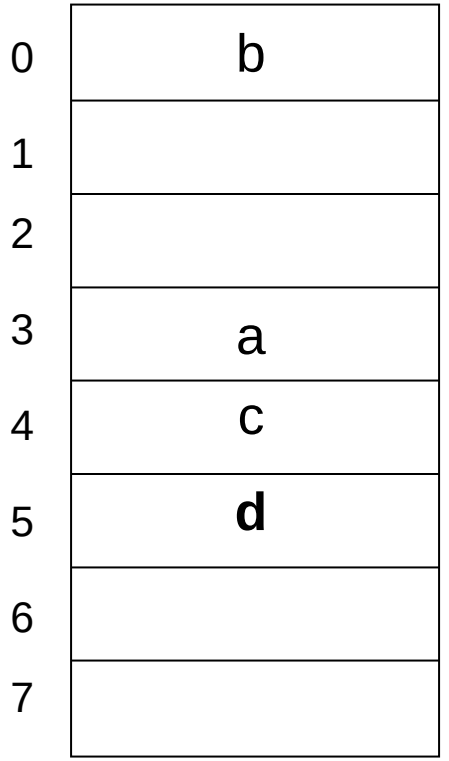
- How can we solve the problem of collisions?
  - **Solution-1** Closed hashing, a.k.a. open addressing
  - **Solution-2** Open hashing, a.k.a. separate chaining
- Difference has to do with whether collisions are stored *outside the table* (open hashing) or whether collisions result in storing one of the records at *another slot in the table* (closed hashing)

Closed Hashing (Open Addressing)

- Associated with closed hashing is a rehash strategy:
  - "If we try to place ***x*** in bucket ***h(x)*** and find it occupied, find alternative location ***h1(x)***, ***h2(x)***, etc. Try each in order, if none empty table is full,"
- ***h(x)*** is called **home bucket**
- Simplest rehash strategy is called linear hashing
  - $h_i(x) = (h(x) + i) \% D$
- In general, our collision resolution strategy is to generate a sequence of hash table slots (probe sequence) that can hold the record; test each slot until find an empty one (probing)

Example Linear (Closed) Hashing

- D = 8, keys *a,b,c,d* have hash values h(a)=3, h(b)=0, h(c)=4, h(d)=3
- Where do we insert d? 3 already filled
- Probe sequence using linear hashing:
  - h1(d) = (h(d)+1)%8 = 4%8 = 4
  - h2(d) = (h(d)+2)%8 = 5%8 = 5*
  - h3(d) = (h(d)+3)%8 = 6%8 = 6
  - etc.
  - 7, 0, 1, 2
- Wraps around the beginning of the table!



Operations Using Linear Hashing

- Test for membership: ***findItem***
- Examine h(k), h_1(k), h_2(k), …, until we find k or an empty bucket or home bucket
- If no deletions possible, strategy works!
- What if deletions?
  - If we reach empty bucket, cannot be sure that **k** is not somewhere else and empty bucket was occupied when **k** was inserted
  - Need special placeholder ***deleted***, to distinguish bucket that was never used from one that once held a value
  - May need to reorganize table after many deletions

Performance Analysis - Worst Case

- **Initialization:** O(b), b# of buckets
- **Insert and search:** O(n), n number of elements in table; all n key values have same home bucket
- No better than linear list for maintaining dictionary!

Improved Collision Resolution

- Linear probing: $h_i(x) = (h(x) + i) \% D$
  - All buckets in table will be candidates for inserting a new record before the probe sequence returns to home position
  - Clustering of records, leads to long probing sequences
- Linear probing with skipping: $\mathbf{h_i(x) = (h(x) + ic) \% D}$
  - **c** constant other than **1**
  - Records with adjacent home buckets will not follow same probe sequence

Reading

- Chapter 12 “Hash Tables” of book “Introduction to Algorithms” By Thomas H. Cormen et al